Updating Orchestration JS

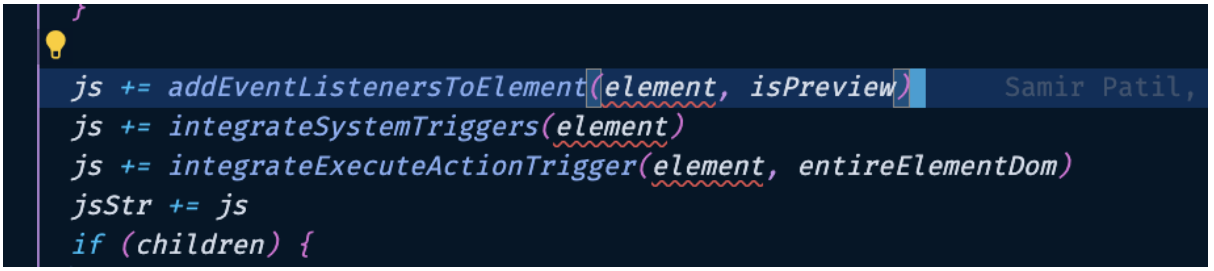
Metadata

Authors: Thejas Narasimhan
Project URL:
Project Summary: Upgrading orchestration js

Abstract

The way orchestration JS working right now is

1. For any element there are 3 layers of js getting added



   a. **addEventListeners**:
      i. This just adds event listeners for 6 hardcoded events like input, click, change, swipeLeft & swipeRight.

```
switch (event) {
  case "input": {
    let str = `
      function (event) {
        transitionApiParams!flowId!.inputParams.${varName} = !element.value!;
        !syncHydrationSourceWithTransitionParamsJs!
        console.log("INPUT: ${varName} ⇒", transitionApiParams!flowId!.inputParams.${varName});
        !element.executions!
        !element.action!
      }
    `
    str = replaceElementActionJsString(element, event, str)
    str = replaceSyncHydrationSourceWithTransitionParamsJsString(
      element,
      isPreview,
      str
    )
    str = replaceElementValueString(element, str)
    str = addElementExecutionsInEvent({
      str,
      event,
      element,
    })
    return str
  }
  case "click": {
    // TODO: Default context params values setting to element specific js
    let str = `
      async function (event) {
        !checkValidationStringWithRevalidate!
        transitionApiParams!flowId!.inputParams.${varName} = true;
        console.log("CLICK : ${varName} ⇒", transitionApiParams!flowId!.inputParams.${varName});
        !optionChoice!
        !element.hydrationInput!
        !element.executions!
        !element.action!
      }
    `
```

      ii. These are all functions created as an event listener. The issue here is these are arranging the function triggers before the action trigger. This kills the whole purpose of adding triggers in an order in LOOP.

      Also code will be cleaner when all the trigger related js are orchestrated in one place.

1. element.executions=> functions/ cardVariables trigger JS
2. Element.action => actions triggers js

b. integrateSystemTriggers

i. This function has a map for all the system trigger related js. So whatever event this trigger is attached to, a new event listener is created for the event again

```
case "exitFlow": {
  const logic = `
    _sdk_function_postMessage(JSON.stringify({type:"exit-flow", data: "", source:podId}));
  `
  return `
    var ${varName}!flowId!${name} = document.getElementById('${id}');
    ${varName}!flowId!${name}.addEventListener('${event}', () => {
      ${wrapWithExecutionCondition(executionCondition, logic)}
    });
  `
}

case "redirectFlow": {
  const logic = `
    _sdk_function_postMessage(JSON.stringify({type:"redirect-event", data: {redirectUrl:${redirectUrl}, data:transitionApiParams!flowId!}, source:podId}));
  `
  return `
    var ${varName}!flowId!${name} = document.getElementById('${id}');
    ${varName}!flowId!${name}.addEventListener('${event}', () => {
      ${wrapWithExecutionCondition(executionCondition, logic)}
    });
  `
}

case "transition": {
  const logic = `
    _sdk_function_postMessage(JSON.stringify({type:messageType!flowId!, data:transitionApiParams!flowId!, source:podId}));
  `
  let str = `
    var ${varName}!flowId!${name} = document.getElementById('${id}');
    ${varName}!flowId!${name}.addEventListener('${event}', async () => {
      !checkValidationStringWithRevalidate!
      ${wrapWithExecutionCondition(executionCondition, logic)}
    });
  `
  return replaceValidationString(runValidations, str)
}
```

c. integrateExecutionActionTrigger

i. This creates executeAction js for each of the action in the element triggers.

ii. This also creates redundant event listeners and also creates uncertainty on which will be triggered first

```
let sdkFunction = `
  _sdk_function_postMessage(JSON.stringify({
      type:"execute-action", data:transitionApiParams!flowID!, source:podId, callerElementId: "${strippedId}"
    })
  );
`

let str = `
  var ${varName}__${strippedId}__${strippedName} = document.getElementById('${id}');
  ${varName}__${strippedId}__${strippedName}.addEventListener('${event}', async () => {
    // Early return logic to prevent action execution
    !checkValidationStringWithRevalidate!

    ${wrapWithExecutionCondition(executionCondition as string, "!sdkFunction!")}
  });
`

str += `
  // DO NOT CHANGE THE Function name, we are calling it in the window.onmessage handler
  function onExecuteActionRunHydration_${strippedId}(actionResponse) {
    ${JSON.stringify(Object.values(body[name].resVarMap))}.forEach((key) => {
      window[key] = actionResponse[key];
    });

    transitionApiParams!flowID! = {
      actionResponse,
      ...transitionApiParams!flowID!,
      inputParams:{
        ...transitionApiParams!flowID!.inputParams,
        ...actionResponse,
        action:[]
```

Changes proposed

Very small set of changes

1. For element js no need for (b)integrateSystemTriggers, (c)integrateExecutionActionTrigger. We can add only eventListeners and these (b) and (c)will be part of the triggers associated with the element
2. These will be eventSpecific JS that we have as different for different events.So we can start with this js for each of these events.

```
export const eventSpecificJs = (event: string) => {
  switch (event) {
    case "input":
      return `transitionApiParams!flowId!.inputParams.!varName! = !element.value!;
              !syncHydrationSourceWithTransitionParamsJs!
              console.log("INPUT: !varName! =>", transitionApiParams!flowId!.inputParams.!varName!);
              !element.action!`

    case "click":
      return `!checkValidationStringWithRevalidate!
              transitionApiParams!flowId!.inputParams.!varName! = true;
              console.log("CLICK : !varName! =>", transitionApiParams!flowId!.inputParams.!varName!);
              !optionChoice!
              !element.hydrationInput!
              !element.action!
              `
    case "change":
      return `transitionApiParams!flowId!.inputParams.!varName! = !element.value!;
              console.log("CHANGE : !varName! =>", transitionApiParams!flowId!.inputParams.!varName!);`
    case "submit":
      return `transitionApiParams!flowId!.inputParams.!varName! = !element.value!;
              !syncHydrationSourceWithTransitionParamsJs!
              console.log("SUBMIT: !varName! =>", transitionApiParams!flowId!.inputParams.!varName!);`
    case "swipeLeft":
      return `transitionApiParams!flowId!.inputParams.swipeLeft_!varName! = true;
              console.log("swipeLeft_!varName! =>", transitionApiParams!flowId!.inputParams.swipeLeft_!varName!);`
    case "swipeRight":
      return `transitionApiParams!flowId!.inputParams.swipeRight_!varName! = true;
              console.log("swipeRight_!varName! =>", transitionApiParams!flowId!.inputParams.swipeRight_!varName!);`
    case "timeEvent":
      return `console.log("TIME EVENT: !varName!");`
  }
}
```

3. Since a lot of html elements do not support “load” event listeners, for load events we can have different listener.

```
events
  .filter((e) => e !== "load")
  ?.map((event) => {
    eventListenersString += `
     element${name}!flowId!${strippedId}.addEventListener("${event}",${eventHandlerJs(event, element, isPreview, entireElementDom)});`
  })
if (events.includes("load")) {
  eventListenersString += `
      document.addEventListener("DOMContentLoaded", (event) => {
        if(document.getElementById('${id}')) {
          (${eventHandlerJs("load", element, isPreview, entireElementDom)})();
        }
      });`
}
```

4. Standard set of replacement of js strings we do

```
let str = eventSpecificJs(event) || ""

//Replace Element Action JS
str = replaceElementActionJsString(element, event, str)

//Replace Sync Hydration JS
str = replaceSyncHydrationSourceWithTransitionParamsJsString(
  element,
  isPreview,
  str
)

//Replace element Value JS
str = replaceElementValueString(element, str)

//Replace HydrationInput JS
let hydrationInput = element?.data?.hydrationInput || {}
let hydrationInputStr = ""
if (hydrationInput) {
  Object.keys(hydrationInput).map((key) => {
    const value = hydrationInput[key]
    if (value !== "") {
      hydrationInputStr += `transitionApiParams!flowId!.inputParams.${key} = ${value};`
    }
  })
}
str = str.replace("!element.hydrationInput!", hydrationInputStr)

//Replace Validations JS
str = replaceValidationString(element?.data?.runValidations, str)
```

5. Trigger JS will be added for this element's for the given specific events

```
//Trigger JS
str += getTriggerJs(element, event, entireElementDom)

//Replace VarNames
str = str.replace(/!varName!/g, varName)

str = str.replace(/"!/g, "").replace(/!"/g, "")

//Final EventListener function
return `async function (event){
  ${str}
}`
```

6. TriggerJs will orchestrate the js for each of the triggers added in sequence

```
export const getTriggerJs = (        You, 20 hours ago • Works until c
  element: Element,
  event: string,
  entireElementDom: CardDOM
) => {
  let eventTriggers =
    element?.executions?.triggers?.filter(
      (trigger: ElementTrigger) => trigger.event === event
    ) || []

  eventTriggers = eventTriggers.filter(
    (trigger: ElementTrigger) =>
      trigger.event &&
      (!(trigger.eventHandlerType === "system") ||
        (trigger.eventHandlerType === "system" && trigger.body))
  )

  let triggersJs = ""
  let actionTriggerCompleted = false
```

7. Each of the trigger will get its own expressions to execute

```
if (trigger.eventHandlerType === "system") {
  triggerJs = systemTriggersHandlerMap(
    {
      ...(trigger.body as ElementSystemTrigger),
      event: trigger.event,
      executionCondition: trigger.executionCondition,
    },
    element
  )
} else if (
  trigger.eventHandlerType === "action" &&
  !actionTriggerCompleted
) {
  triggerJs = integrateExecuteActionTriggerByEvent(
    element,
    event,
    entireElementDom
  )
  actionTriggerCompleted = true
} else if (trigger.eventHandlerType == "function" && trigger.body) {
  triggerJs = addDisableInputWrapperJs(
    singleCustomFunctionCallJs(trigger.body)
  )
} else if (trigger.eventHandlerType === "cardState" && trigger.body) {
  triggerJs = cardStateTriggerHandlerMap({ ...trigger.body }, element)
}
```

8. Each of these triggers will be wrapped in an async function that is being awaited to complete. So that all these triggers will be in sync. Also they are wrapped in try catch to avoid js breaking

```
triggersJs +=
  //@ts-ignore
  ` //Trigger: ${trigger?.body?.name || trigger?.eventHandlerType}
                try{
                    await (async function(){
                    ${triggerJs}
                    })();
                }
                catch(e){
                    console.log("Error in system trigger: ${trigger?.body?.name} for element: ${element.id}")
                }
    `
```

This approach ensures that the trigger js are all wrapped in one single event listener for the element. Making the js cleaner.